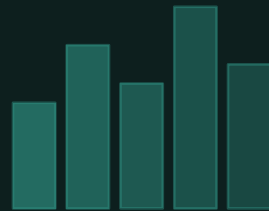


noCommerce + OpenTelemetry



What this presentation covers

01

Flow selected

Customer places an order — end-to-end through Basket, Payment, Inventory

02

Instrumentation decisions

Span boundaries, metric choices, and the reasoning behind each

03

Privacy strategy

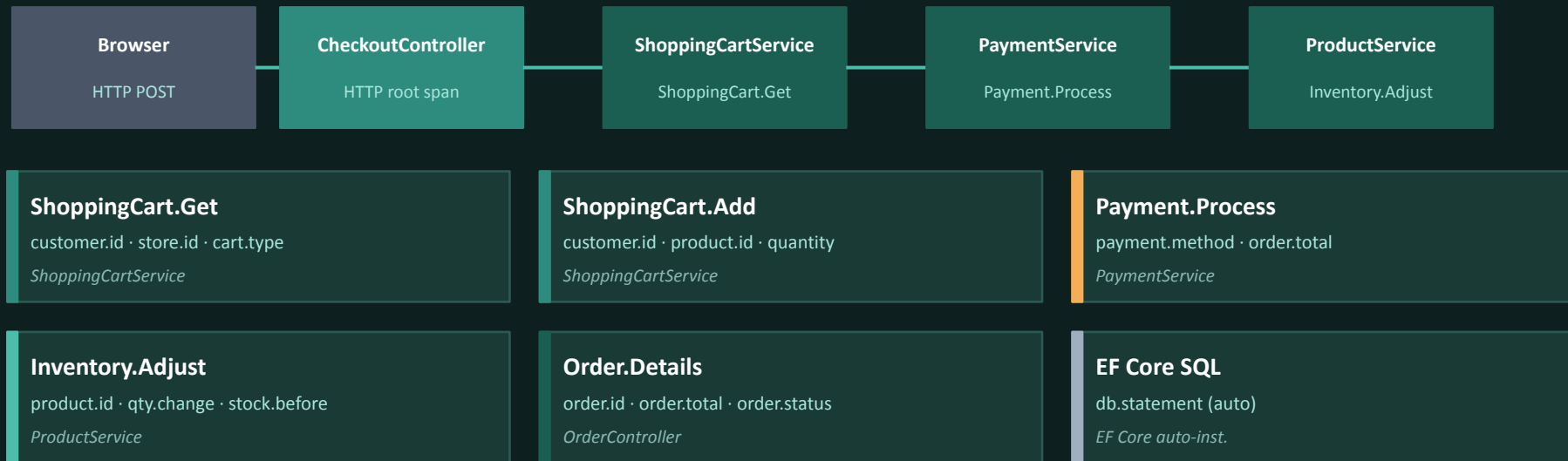
What was excluded, where redaction happens, and why that layer

04

Tradeoffs

What was not instrumented, what would change architecturally

Customer places an order — end-to-end trace



Custom metrics — and why each one matters

orders.started

Counter

If this counter flatlines while HTTP traffic continues, customers are reaching checkout but the pipeline is not starting — a session or validation failure upstream, visible before any user reports it.

If this spiked or dropped at 2am, would an engineer know what to do? → Yes — page the payments team immediately.

order.value

Histogram

Goes beyond request count: shows revenue distribution, flags anomalous values (£0.00 = pricing bug), and supports SLOs on checkout revenue throughput, not just request throughput.

What would a £0.00 spike tell you? → Coupon or pricing logic is broken — silent without this metric.

orders.failed

Counter

The ratio failed/started is the checkout conversion failure rate — the most commercially significant metric in an e-commerce platform. A spike means the payment gateway is degrading.

What makes this more than just error_rate? → It is scoped to the order pipeline specifically, not all HTTP errors.

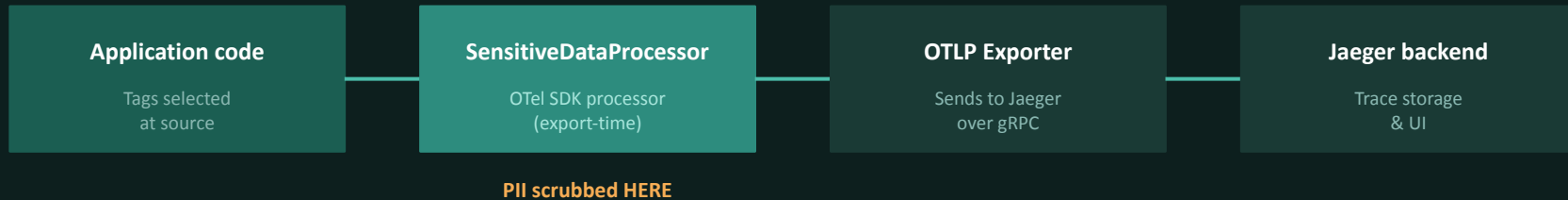
inventory.level

ObservableGauge *

Surfaces stockout risk before customers see "Out of stock". Enables alert rules: stock < 5 on high-velocity products triggers a restocking alert. (* currently Histogram — type fix pending.)

Why Gauge, not Histogram? → Stock is a point-in-time scalar, not a distribution.

Privacy strategy — where redaction happens and why



Fields excluded from spans

email	Direct personal identifier
password	Credential
credit / card	Payment PCI data
address	Billing / shipping PII
phone / ssn	Personal identifiers
payment	Catch-all for payment fields

Why the SDK processor layer?

Centralised

One processor covers the entire pipeline. No per-span discipline required.

Defence-in-depth

Even if a developer tags `customer.email` by mistake, it is scrubbed before leaving the process.

Not at the collector

Collector-side redaction requires running a Collector — unnecessary complexity here.

Not at storage

Data-at-rest redaction is too late. The data was already transmitted in plaintext.

Tradeoffs

Gap

IEventPublisher context propagation

Consumer spans (email dispatch, inventory reservation after order placed) appear as disconnected root spans in Jaeger. Fix is 5 lines in Nop.Core — capture Activity.Current before the dispatch loop, restore inside each consumer invocation. Not yet implemented.

Deliberate

Service class internals — not instrumented

OrderProcessingService is 2,000+ lines. Instrumenting internal steps would produce noisy spans without adding operational value. Boundary instrumentation (entry/exit of the service method) tells the same story at lower cost and zero risk to business logic.

Known gap

Caching layer opacity

Cache hits and misses are invisible. A slow trace cannot be distinguished as 'cache miss + slow query' vs 'always slow query'. Instrumenting the cache service boundary would close this — scoped to future work.

Demo

Key takeaways

Instrument boundaries, not internals.

Architecture first

Understanding the Onion layers defined where instrumentation was possible and where it was not.

Metrics must be actionable

`orders.failed/orders.started` at 2am = call the payments team. That is the bar every metric was held to.

Privacy is structural

One SDK processor at export time is safer than per-span discipline at every developer touchpoint.

Gaps are honest

`IEventPublisher` context propagation is unimplemented. Named, documented, and queued — not hidden.